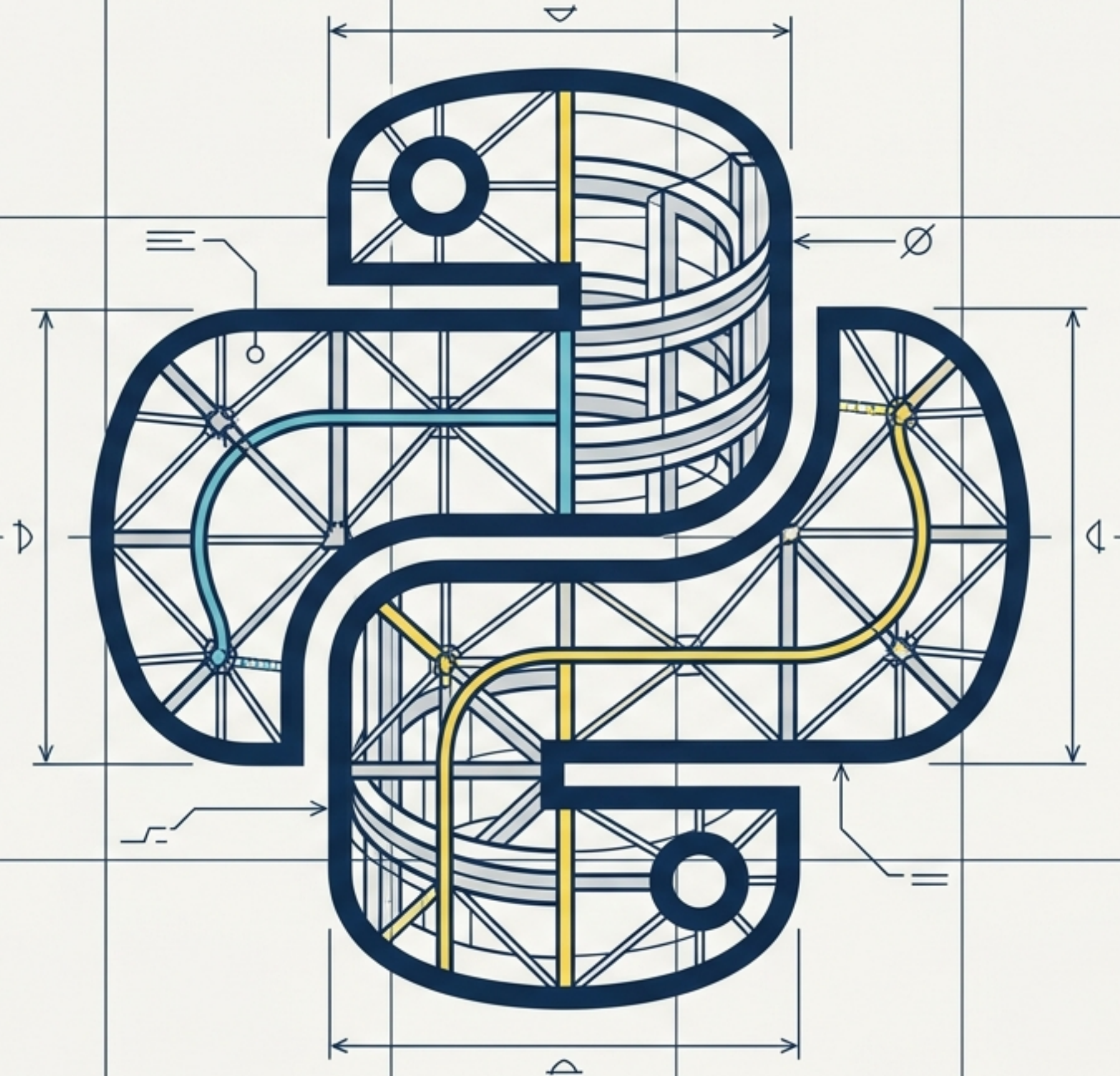
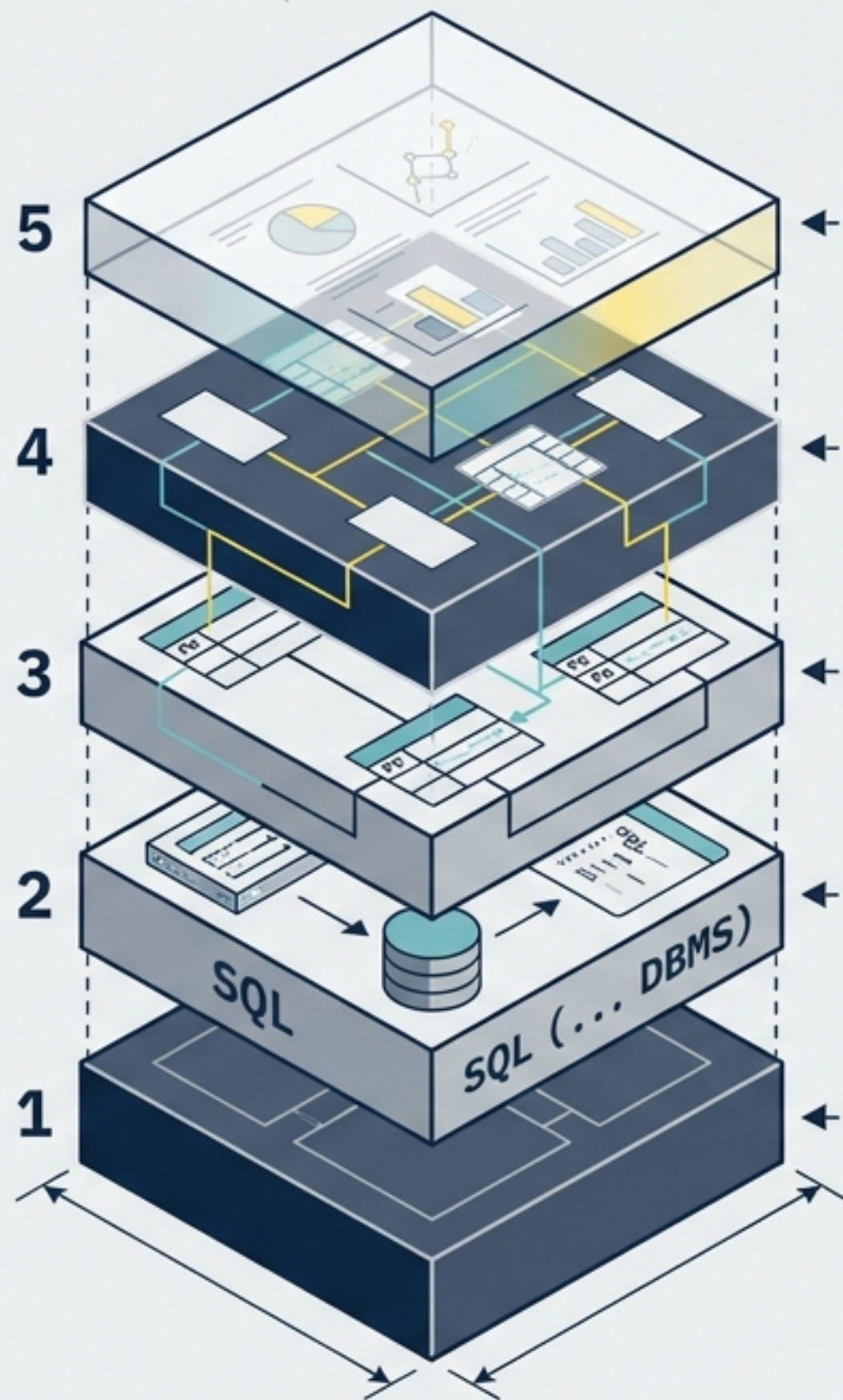


USING DATABASES WITH PYTHON: A COMPREHENSIVE BLUEPRINT

From Object-Oriented
Application Logic to
Relational Schemas and
Data Visualisation





**Layer 5: Insight Extraction
(Data Mining & Dataviz)**

The interface bringing insights to light.

**Layer 4: Complex Linkages
(M:N Relationships)**

The wiring connecting disparate entities.

**Layer 3: Structural Integrity
(Relational Schemas)**

The floorplan ensuring data normalisation.

**Layer 2: Storage Engine
(SQL & DBMS)**

The foundation for data persistence and retrieval.

**Layer 1: Application Logic
(OOP)**

Key Insight: This blueprint maps isolated concepts—from Python **inheritance** to database **junction tables**—into a single, **production-ready application architecture**.

Layer 1 — The Four Pillars of Object-Oriented Python

Encapsulation

Bundles data and methods; restricts direct access to internal state.

```
self.__balance
```

Abstraction

Hides implementation complexity; exposes only essential interfaces via abstract classes.

```
from abc import ABC, abstractmethod
```

Inheritance

Enables a child class to reuse and extend parent functionality (The 'IS-A' relationship).

```
class Dog(Animal):  
    super().__init__()
```

Polymorphism

Allows the same method to behave differently across object types (Method overriding & Duck typing).

```
animal.sound()
```

OOP in Production – Protecting State & Defining Contracts

```
class BankAccount:
    def __init__(self, owner, balance):
        self.owner = owner
        → self.__balance = balance

    def deposit(self, amount):

    def deposit(self, amount): ←
        if amount > 0:
            self.__balance += amount
```

```
from abc import ABC, abstractmethod

class Vehicle(ABC):
    @abstractmethod ←
    def start(self):
        pass
```

- **Name Mangling in Action:**
Creates a protected internal state to prevent uncontrolled external modification.

- **State Protection:**
External code must pass through this validated getter/setter method.

Enforcing the Contract: •
Subclasses are legally obligated to provide concrete implementations, or Python raises a TypeError.

Layer 2 — The Relational Database Engine



Oracle	Enterprise-scale, commercial. For mission-critical systems.
MySQL / PostgreSQL	Open-source, highly reliable. Broad web adoption and advanced extensibility.
SQL Server	Microsoft's RDBMS. Tightly integrated with Windows tools.
SQLite	Serverless, file-based, and embedded directly into applications.



The Embedded Choice: Ideal for Python prototyping without heavy infrastructure.

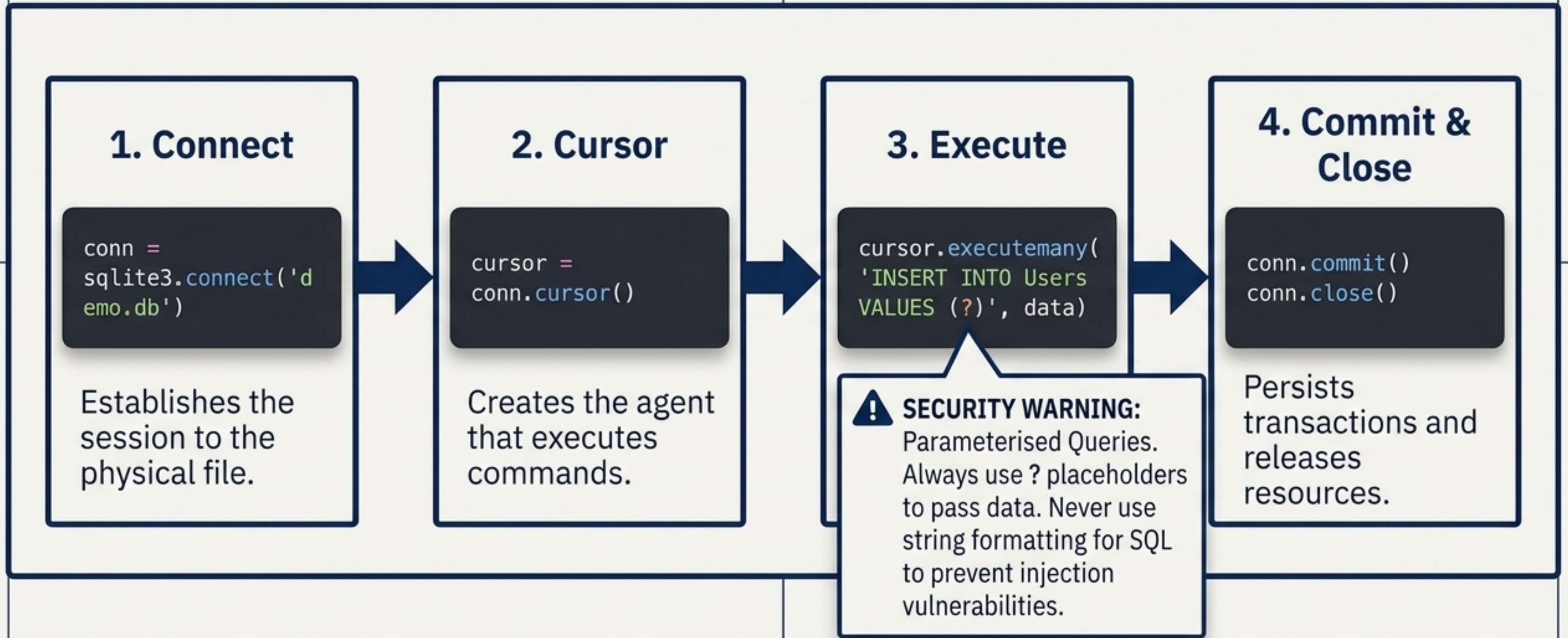
Tooling Note:
DB Browser for SQLite (DB4S) is the essential open-source graphical tool for visual database management.

The Taxonomy of SQL

DDL Data Definition	DML Data Manipulation	DQL Data Query	TCL Transaction Control	DCL Data Control
Define structure. CREATE, ALTER, DROP	Manage data. INSERT, UPDATE, DELETE	Retrieve data. SELECT	Manage transactions. COMMIT, ROLLBACK	Control access. GRANT, REVOKE

Key Insight: SQL is a declarative language. You specify **WHAT** data you need; the database management system determines **HOW** to execute the retrieval.

Integrating Python and SQLite



Layer 3 – Database Design & The Golden Rule

The Flat Table Bloat

Track	Album
Come Together	Abbey Road
Something	Abbey Road
Maxwell's Silver Hammer	Abbey Road
Oh! Darling	Abbey Road
Octopus's Garden	Abbey Road

The Relational Model

Album

id	title
1	Abbey Road

Foreign Key

Track

title	album_id
Come Together	1
Something	1
Maxwell's Silver Hammer	1
Oh! Darling	1
Octopus's Garden	1

The Golden Rule: Do not store the same string data in multiple places.
One thing in the real world = one record in the database.

Primary Key: Auto-incremented integers (e.g., id) uniquely identifying the row.

Foreign Key: A column in the child table referencing the parent table's Primary Key.

The Power of JOINS

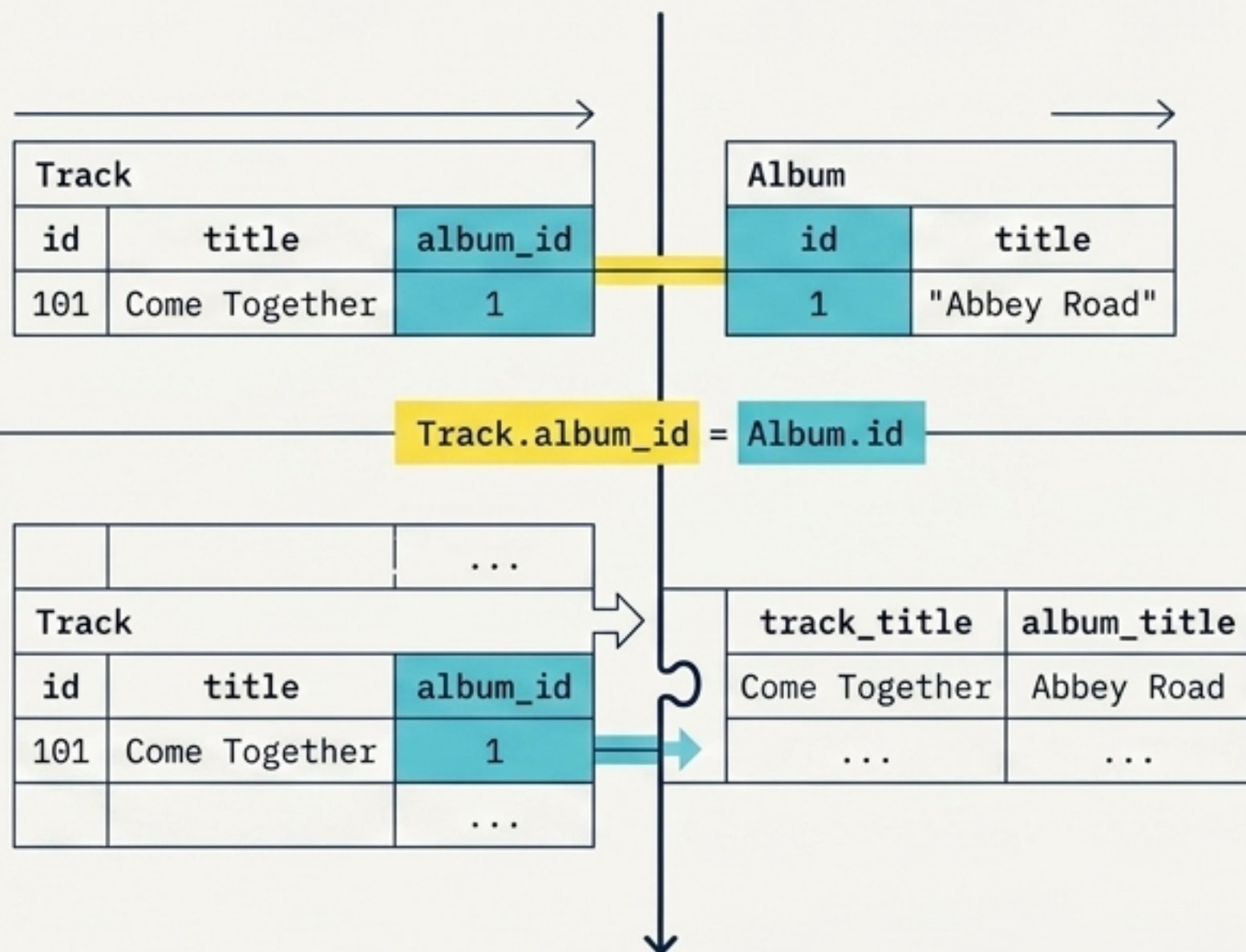


Table-Qualified Columns:
Prevents ambiguity when both tables have a 'title' column.

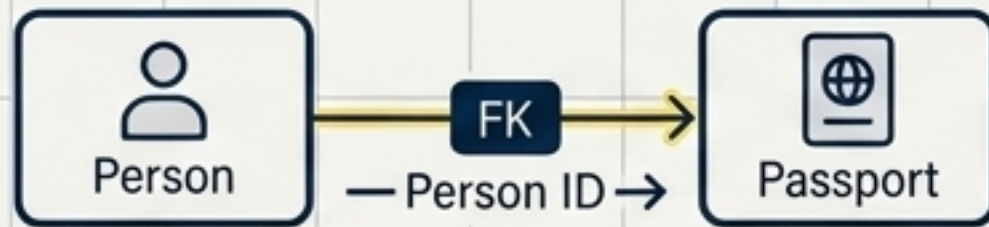
Aliases: Renames output columns for programmer readability.

```
SELECT
    Track.title AS track_title,
    Album.title AS album_title
FROM Track
INNER JOIN Album ON Track.album_id = Album.id
```

The mechanism that seamlessly combines records based on matching foreign key logic, without data duplication.

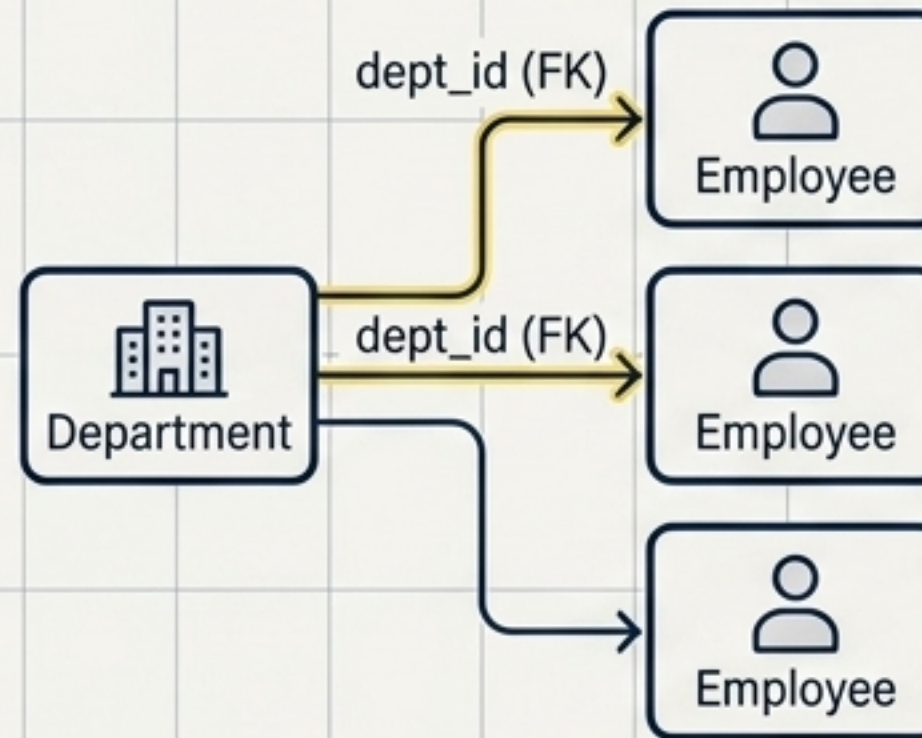
Layer 4 — Demystifying Database Relationships

One-to-One (1:1)



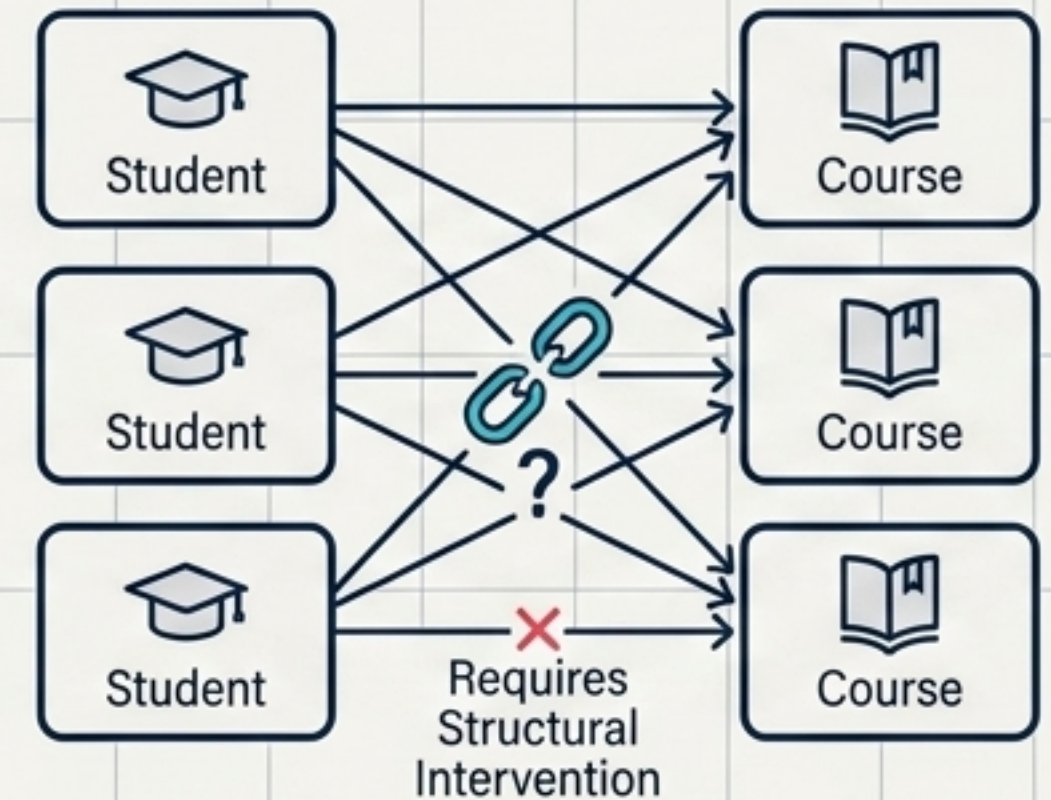
Rule: Foreign Key can technically go in either table, but usually resides in the subordinate entity.

One-to-Many (1:N)



Rule: The Foreign Key always goes on the “Many” side. (The Employee table holds dept_id).

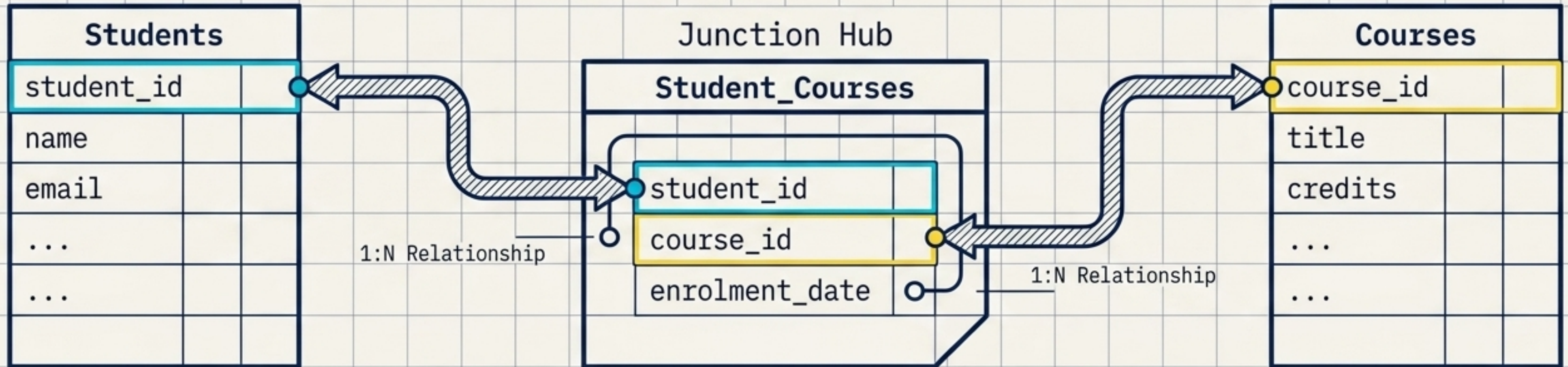
Many-to-Many (M:N)



Rule: Cannot be resolved with a simple Foreign Key in either parent table. Requires structural intervention.

Resolving Many-to-Many Connections

The Junction Table Schematic



The Concept

Decompose the M:N relationship into two separate 1:N relationships using a central bridge table.

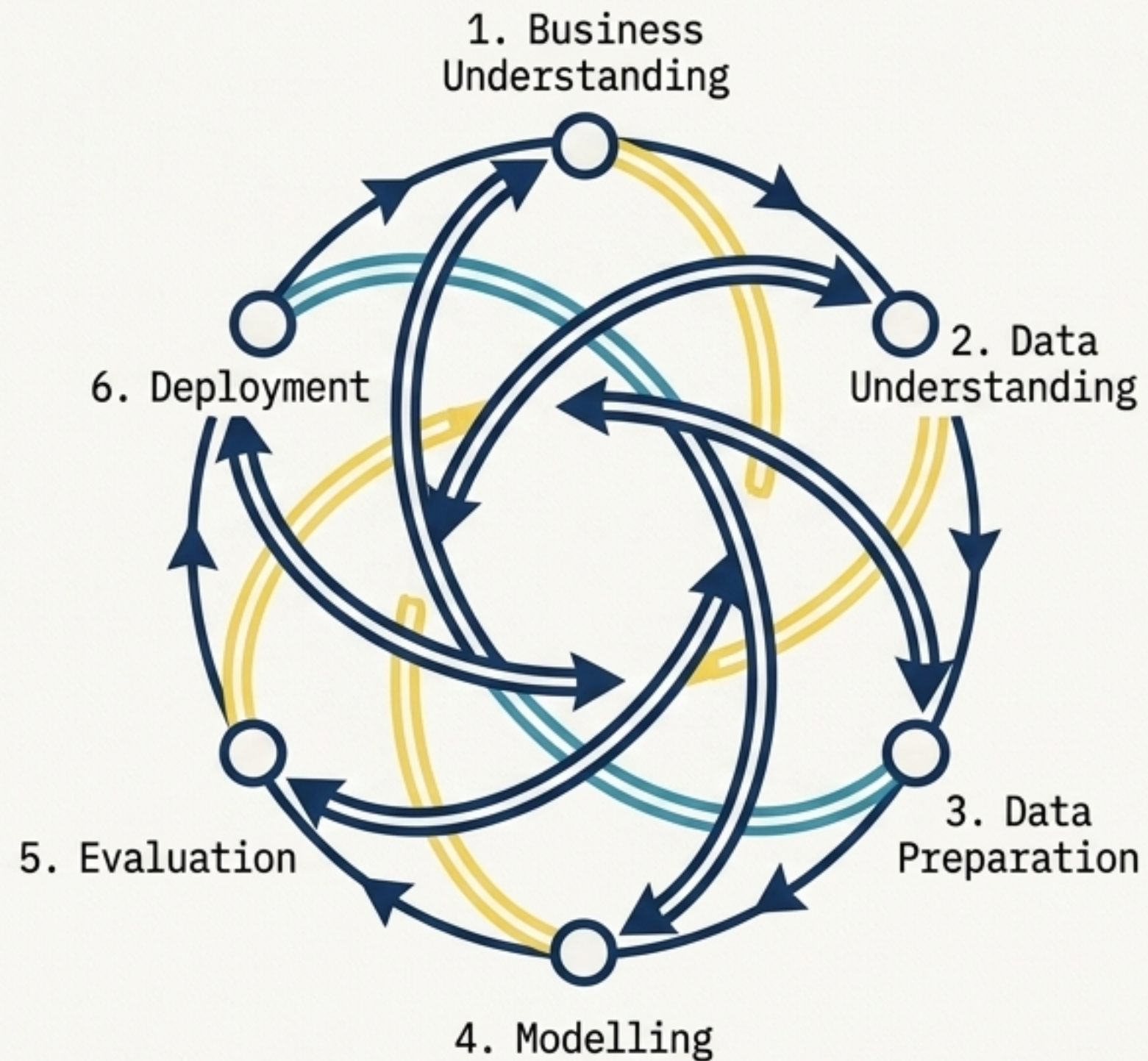
Composite Primary Key

The combination of (student_id, course_id) ensures a student cannot enrol in the same course twice.

Relationship Metadata

The junction table holds data specific to the association itself, such as enrolment_date.

Layer 5 – Data Mining & The CRISP-DM Process



Querying vs. Mining

A SQL SELECT statement retrieves known facts.
Data Mining computationally discovers previously unknown, non-trivial patterns.

Key Techniques

Classification:
Categorising data
(e.g., spam detection).

Clustering: Grouping
by similarity without
predefined labels.

Association Rules:
Discovering co-
occurrence (market
basket analysis).

Anomaly Detection:
Identifying deviations
(fraud detection).

SQL Aggregation for Business Intelligence

```
SELECT
  category,
  COUNT(*) (SQLite AS num_sales,
  ROUND(SUM(amount) (SQLite, 2) AS
  AS total_revenue
FROM Sales (SQLite)
GROUP BY category
ORDER BY total_revenue DESC
```



Category	# Sales	Total Revenue
Electronics	85	\$45,000
Apparel	120	\$28,500
Home Goods	60	\$18,200

The engine partitions all rows into distinct subsets based on the 'category' value.

Aggregate functions are applied independently to each subset, transforming thousands of raw transaction rows into actionable business intelligence.

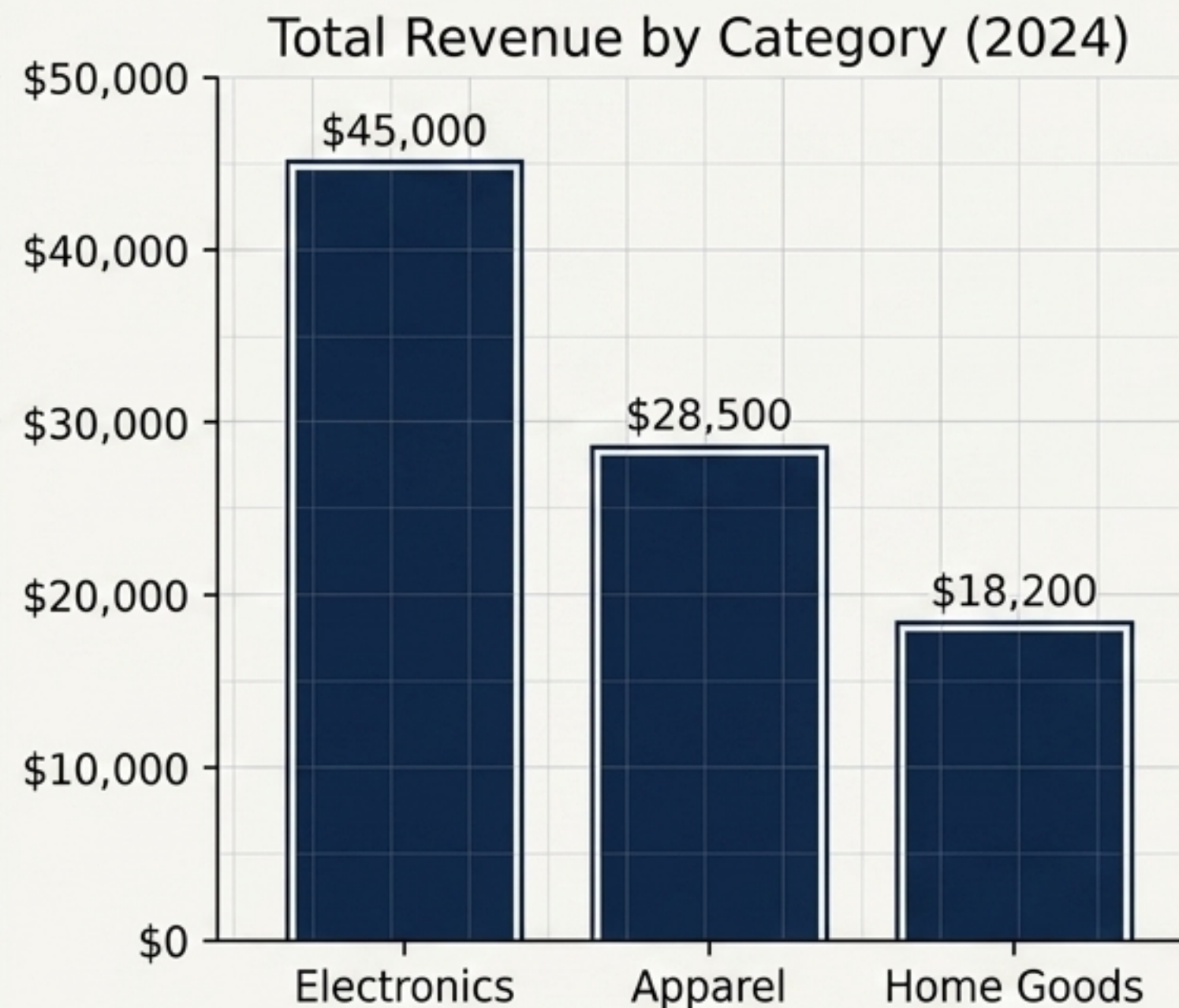
From Database to Visualisation

```
1 fig, ax = plt.subplots()
2
3 ax.yaxis.set_major_formatter(mticker.FuncFormatter(...))
4
5 ax.spines[['top', 'right']].set_visible(False)
6
```

Object-Oriented API: Precise control over the figure canvas.

Data Formatting: Dynamically translating raw floats into currency strings.

Removing Chart Junk: Stripping unnecessary borders to adhere to best practices.



The Complete Blueprint

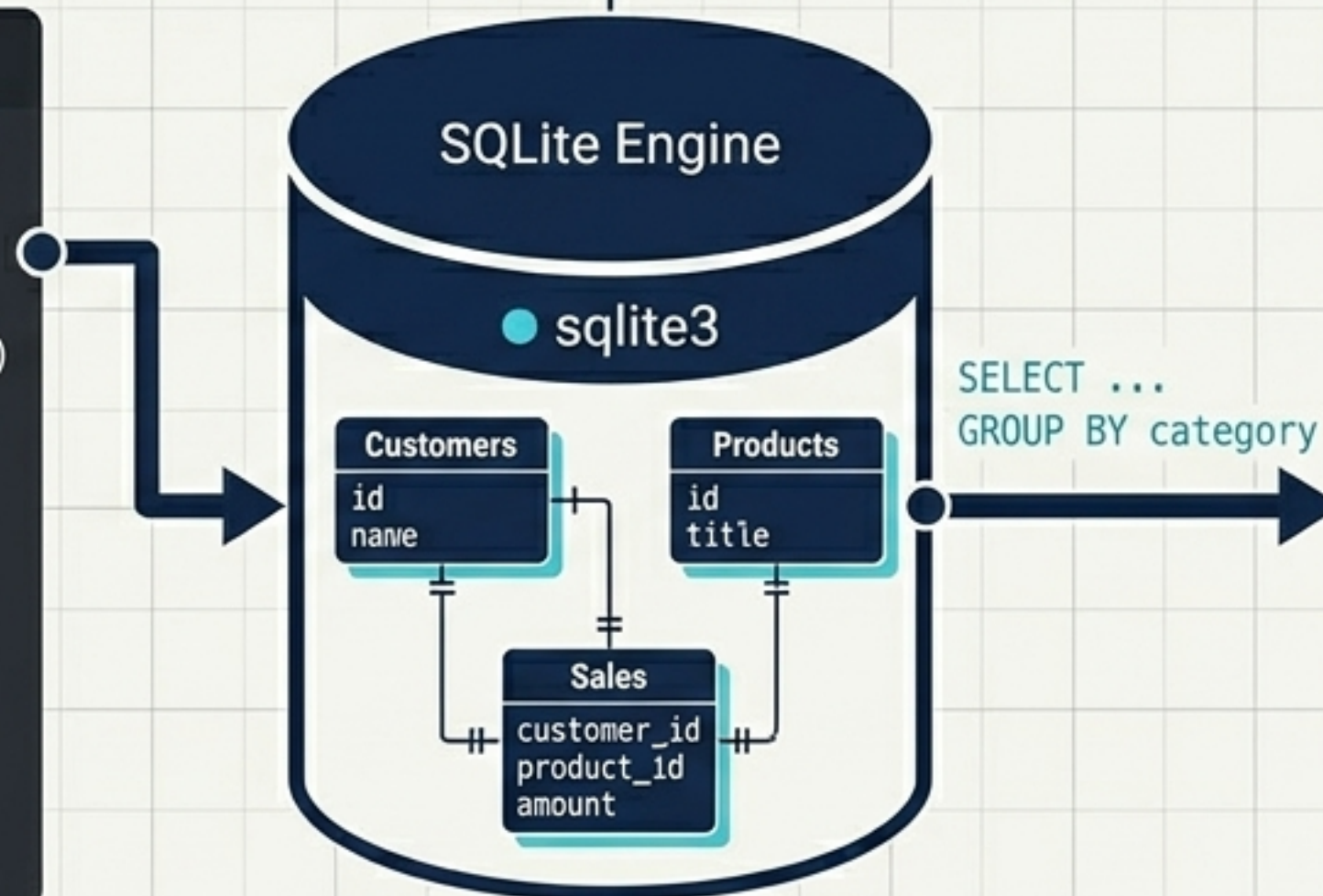


OOP Class Architecture

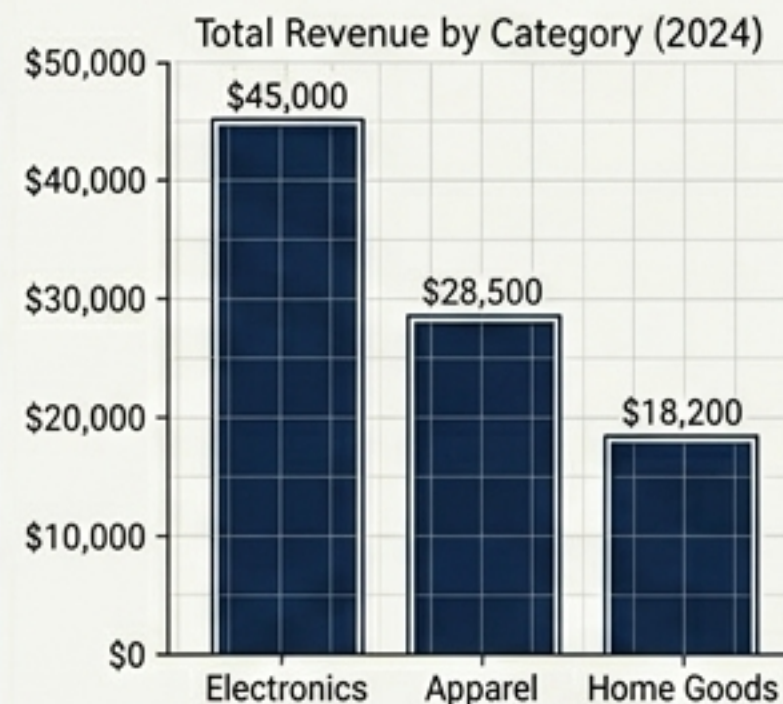
```
class DataPipeline:
    def __init__(self, db_path):
        self.conn = sqlite3.connect(db_path)
        self.cursor = self.conn.cursor()

    def fetch_data(self, query):
        self.cursor.execute(query)
        return self.cursor.fetchall()

    def visualize(self, data):
        # ... (Matplotlib Logic)
        pass
```



Total Revenue by Category
(Aggregated Data)



Closing Insight: You are no longer just writing scripts. You are architecting full-stack, data-driven pipelines—from structured application logic to relational integrity, culminating in visual insight.